

Pass@k: Measuring Agents that Retry, by Hand

What a leaderboard number really counts when a model gets k tries — derived two ways, verified to the last binomial coefficient.

What this document does. It takes the one metric Module 3.7 keeps warning you about — `pass@k`, “the model gets k attempts; at least one must pass” — and computes it by hand for a single tiny code-agent problem. We fix a per-attempt success probability, build `pass@k` from the complement, then derive the *unbiased* estimator that HumanEval/Codex actually report ($1 - \binom{n-c}{k} / \binom{n}{k}$) and run it on a concrete sample ($n=10$ generations, $c=3$ correct). Every figure below — including the module’s claim that a model at `pass@10`≈95% can sit at `pass@1`≈70% — is reproduced by the Python program in Appendix A using *exact* integer binomials, so the arithmetic is exact and self-consistent.

Where this sits. This is **Topic 1 of Module 3.7** (Benchmark Suites for Agents) — the formal core behind the module’s “pass@k Explained” widget and Pitfall 04, *pass@k inflation*. Its companions: Module 3.1 (the agent-suite cards — SWE-bench, WebArena, GAIA, τ -bench) and the same module’s §08 audit checklist, whose second question is simply “pass@1 or pass@k? What k ?”

Concepts covered, in order: one attempt as a Bernoulli trial · `pass@k` from the complement $1 - (1 - p)^k$ · the with-replacement intuition · the *unbiased* sampling estimator $1 - \binom{n-c}{k} / \binom{n}{k}$ · drawing *without* replacement and why it differs · diminishing returns and the $(1 - p)$ decay · `pass@k` as optimism vs. `pass@1` as reliability (τ -bench’s `pass~k`) · why contamination inflates `pass@1` and eval sets must be frozen.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one problem, one model, k tries

Take a single task from a code benchmark — say one HumanEval-style problem with a hidden test. Our code agent is *stochastic*: sampled at temperature, each independent attempt either passes the hidden test or fails. A benchmark with `pass@k` scoring lets the agent make k attempts and scores the problem *solved* if *at least one* attempt passes. That is the whole object we are going to measure. Everything else is counting.

The quantities. We give every symbol a small, round value so the sums are followable on paper. The mechanics are identical at any scale — only the constants change.

Quantity	Symbol	Value here
Success probability of one attempt	p	0.30
Number of attempts the metric allows	k	1, 2, 3, 4, 5
Samples generated to <i>estimate</i> the score	n	10
of which passed the hidden test	c	3
Implied per-sample rate from the sample	c/n	0.30

Notation. $\text{pass}@k$ is the probability (or estimated fraction) that the problem is solved within k attempts. $\binom{a}{b}$ is the binomial coefficient “ a choose b ,” the number of ways to pick b items from a without regard to order, with $\binom{a}{b} = \frac{a!}{b!(a-b)!}$ and $\binom{a}{b} = 0$ when $b > a$. We carry four decimals.

Intuition

pass@k answers “if I let the agent retry k times and keep the best, how often does *something* land?” It rewards a single lucky attempt. That makes it the natural metric for a deployment that *can* retry — a coding loop that runs the tests and tries again — and a *misleading* one for a deployment that cannot. The whole of Module 3.7’s Pitfall 04 is: do not quote **pass@10** if production gets one shot.

2 Step 1 — $\text{pass}@k$ from the complement (the simple model)

Assume the k attempts are independent, each succeeding with probability p . Computing the probability of “at least one success” directly means summing over $1, 2, \dots, k$ successes — annoying. The trick is to count the *only* way to fail: *every* attempt must miss. A single attempt fails with probability $(1 - p)$; k independent attempts all fail with probability $(1 - p)^k$. So

$$\text{pass}@k = P(\geq 1 \text{ success in } k) = 1 - P(\text{all } k \text{ fail}) = 1 - (1 - p)^k. \quad (1)$$

With $p = 0.30$, so $(1 - p) = 0.70$, we just take powers of 0.70 and subtract from one:

k	$(1 - p)^k = 0.7^k$	$\text{pass}@k = 1 - 0.7^k$	marginal gain
1	0.7000	0.3000	+0.3000
2	0.4900	0.5100	+0.2100
3	0.3430	0.6570	+0.1470
4	0.2401	0.7599	+0.1029
5	0.1681	0.8319	+0.0720

Sample check, $k = 3$: $0.7^3 = 0.343$, so $\text{pass}@3 = 1 - 0.343 = 0.657$. ✓ A model that succeeds only 30% of the time on a single try clears the problem **83%** of the time given five tries. The number went up without the model getting any better — it just got more lots.

Mini-example — this operation in isolation

Why the complement and not a direct sum? “At least one success” is the union of many disjoint cases (exactly 1, exactly 2, ...). “All fail” is a *single* event, and independence makes its probability a clean product $(1 - p)^k$. The same move powers “at least one shared birthday” and “at least one heads in k flips.” Counting the one way to lose is almost always easier than counting the many ways to win.

3 Step 2 — diminishing returns and the $(1 - p)$ decay

Look at the last column above: each extra attempt buys *less* than the one before. The gain from attempt k is

$$\Delta_k = [1 - (1 - p)^k] - [1 - (1 - p)^{k-1}] = (1 - p)^{k-1} - (1 - p)^k = p(1 - p)^{k-1}.$$

So $\Delta_1 = p = 0.30$, and every subsequent gain is exactly $(1 - p) = 0.70$ times the previous one: $0.30 \rightarrow 0.21 \rightarrow 0.147 \rightarrow 0.1029 \rightarrow 0.072$. ✓ The gains form a geometric decay with ratio 0.7. That is the precise statement of “diminishing returns”: the curve climbs toward 1 but never reaches it, and after a handful of tries each additional full run — a full LLM rollout, with all the cost Module 0.0 priced — buys a sliver. The “ k of diminishing returns” is wherever Δ_k drops below what one run is worth to you; here gains are under 10 percentage points by $k = 4$.

Intuition

pass@k is *optimistic by construction*. It credits the problem to you the moment *any* sample works, so it measures “can the model ever do this,” not “will the model reliably do this.” Two models with identical **pass@10** can be miles apart at **pass@1**. That gap is why Module 3.7 insists you “always report **pass@1** alongside any **pass@k**” — **pass@1** is the production-honest number; the rest is headroom.

4 Step 3 — the unbiased estimator benchmarks actually use

Equation (1) needs the true p , which you never know. On a real benchmark you generate n samples for the problem, observe that c of them passed, and must *estimate* **pass@k** from that finite sample. The naive plug-in $1 - (1 - c/n)^k$ is what you get if you pretend each of the k scored attempts is drawn *with replacement* from the pool — as if you could re-draw the same sample twice. The Codex/HumanEval estimator instead asks the cleaner combinatorial question: *if I pick k of my n generated samples without replacement, what is the chance none of them is one of the c correct ones?*

There are $\binom{n}{k}$ equally likely ways to choose k of the n samples. The choices that contain *zero* correct samples are exactly the ways to pick all k from the $n - c$ wrong ones: $\binom{n-c}{k}$ of them. So the probability the chosen k are *all wrong* is the ratio, and **pass@k** is its complement:

$$\widehat{\text{pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \quad (2)$$

Work our sample, $n = 10$, $c = 3$, $k = 5$. The wrong pool is $n - c = 7$.

$$\binom{n-c}{k} = \binom{7}{5} = \frac{7!}{5!2!} = \frac{7 \cdot 6}{2} = 21, \quad \binom{n}{k} = \binom{10}{5} = 252.$$

$$\widehat{\text{pass@}5} = 1 - \frac{21}{252} = 1 - \frac{1}{12} = \frac{11}{12} = 0.9167.$$

✓ The ratio $21/252$ reduces exactly to $1/12 \approx 0.0833$; the estimate is 0.9167. Notice a sanity anchor: at $k = 1$, (2) gives $1 - \binom{9}{1}/\binom{10}{1} = 1 - 7/10 = 0.30 = c/n$ — exactly the per-sample rate, and exactly the simple model’s **pass@1**. The two derivations agree where they must.

Mini-example — this operation in isolation

Why integers, not floats? $\binom{10}{5} = 252$ and $\binom{7}{5} = 21$ are *exact*; their ratio is the exact rational

1/12. Benchmarks compute `pass@k` this way for every problem and average, so rounding errors never accumulate. Tiny-case check of the coefficient: $\binom{7}{5} = \binom{7}{2}$ (choosing the 5 to keep = choosing the 2 to drop) = 21. The symmetry $\binom{a}{b} = \binom{a}{a-b}$ is a free correctness test on every hand computation.

5 Step 4 — with replacement vs. without: why the two estimates differ

Put the two estimators side by side on the same sample $n=10$, $c=3$:

Estimator	Formula at $k = 5$	Value
Naive (with replacement)	$1 - (1 - c/n)^5 = 1 - 0.7^5 = 1 - 0.1681$	0.8319
Unbiased (without replacement)	$1 - \binom{7}{5} / \binom{10}{5} = 1 - 21/252$	0.9167
Difference (unbiased – naive)		0.0847

They disagree by almost 8.5 percentage points, and the unbiased one is *higher*. The reason is the replacement assumption. The naive formula treats each of the 5 drawn attempts as an *independent* re-roll from a 30%-success coin, so all five can “miss” with probability 0.7^5 . But you only generated 10 samples and 3 of them are *already known* to be correct. Drawing 5 of those 10 *without* replacement, it is harder to dodge all three correct ones — in fact with only 7 wrong samples you *can* still draw 5 all-wrong ($\binom{7}{5} = 21$ ways), but it is rarer than the with-replacement model assumes. Fewer all-wrong draws $\Rightarrow$ higher `pass@k`. The unbiased estimator is the honest one: it conditions on the c correct samples you actually have, rather than re-sampling a coin you do not.

Watch out

The naive $1 - (1 - c/n)^k$ is *biased low* and people still quote it because it looks like Equation (1). It is not the same thing: (1) uses the *true* p ; the naive plug-in uses the *estimate* c/n inside a with-replacement model and double-counts the uncertainty. When $n \gg k$ the gap shrinks (sampling with vs. without replacement converge), but on the small n benchmarks actually run, always use Equation (2). It is what HumanEval ships, and it is the only one that returns exactly c/n at $k = 1$.

6 Step 5 — `pass@k` climbs but saturates; `pass@1` is the floor

Tabulate the unbiased estimator across k for our $n=10$, $c=3$ sample, and watch it run up against the ceiling at 1:

k	$\binom{7}{k}$	$\binom{10}{k}$	$\widehat{\text{pass@}k} = 1 - \binom{7}{k} / \binom{10}{k}$	marginal
1	7	10	0.3000	+0.3000
2	21	45	0.5333	+0.2333
3	35	120	0.7083	+0.1750
4	35	210	0.8333	+0.1250
5	21	252	0.9167	+0.0833
6	7	210	0.9667	+0.0500
7	1	120	0.9917	+0.0250

Sample check, $k = 4$: $\binom{7}{4} = 35$, $\binom{10}{4} = 210$, so $1 - 35/210 = 1 - \frac{1}{6} = 0.8333$. ✓ The curve drags $\text{pass@1} = 0.30$ all the way to $\text{pass@7} = 0.99$. This is the module’s headline made literal: **a model that looks ~30% at one shot can look ~92–99% with enough retries** — the same gap the widget illustrates as “ $\text{pass@10} \approx 95\%$ vs. $\text{pass@1} \approx 70\%$.” The shape is invariant; only how fast it saturates depends on c/n . Each step costs one full agent run (Module 0.0), and the marginal column shows the payoff thinning every time — by $k = 5$ you buy 8 points, by $k = 7$ just 2.

Intuition

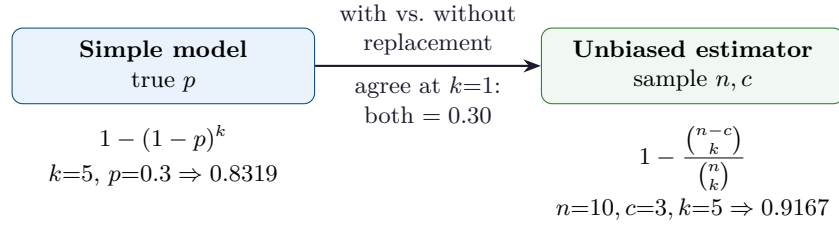
τ -bench (Module 3.1) flips this metric on its head with $\widehat{\text{pass@}k}$: the problem counts only if *all* k runs pass — p^k , not $1 - (1 - p)^k$. $\text{pass@}k$ measures *best-case* (does it ever work); $\widehat{\text{pass@}k}$ measures *worst-case reliability* (does it always work). For a customer-support agent that must not flake, $\widehat{\text{pass@}k}$ is the honest target and $\text{pass@}k$ is the seductive one. Same k , opposite question.

7 A note on saturation and contamination

Two of the module’s six pitfalls fall straight out of this curve. **Saturation:** once frontier models all sit near the ceiling — $\text{pass@1} = 0.99$ on a tiny suite like HumanEval’s 164 problems — the metric stops *discriminating*; every model scores the same 1 and the benchmark is dead as a ranking tool. The fix is harder variants (SWE-bench Verified, LiveCodeBench) or your own domain set.

Contamination is worse because it is invisible. If a benchmark’s problems leak into the model’s training data, the per-attempt rate p you measure is inflated by *memorization*, not capability — and since $\text{pass@1} = p$ for that problem, the leak shows up directly as a too-high pass@1 , which then lifts every $\text{pass@}k$ above it via Equation (1). You cannot tell a memorized $p = 0.9$ from an earned $p = 0.9$ by staring at the score. The only defenses are structural: *freeze* a private test set so it never appears online, and *rotate* a hidden slice over time (the module’s 7-step recipe ends “... version, freeze, rotate”). A frozen, rotated set keeps p honest; a public, static one decays into a memorization check.

8 The two derivations, drawn



Both boxes answer “at least one of k succeeds,” computed as $1 - P(\text{all fail})$. The left draws an unlimited coin (with replacement); the right draws from the finite bag of n samples (without replacement). They meet exactly at $k = 1$ and spread apart as k grows.

9 Pass@k cheat-sheet

Simple model (true p)	$\text{pass}@k = 1 - (1 - p)^k$
Marginal gain of attempt k	$\Delta_k = p(1 - p)^{k-1}$ (geometric, ratio $1 - p$)
Naive plug-in (with replacement)	$1 - (1 - c/n)^k$ (biased low)
Unbiased estimator (Codex)	$\widehat{\text{pass}@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$
Anchor at $k = 1$	both give the per-sample rate (p or c/n)
Reliability metric (τ -bench)	$\widehat{\text{pass}}^k = p^k$ (all k must pass)
Always also report	$\text{pass}@1$ — the production-honest floor

10 An honest word about these numbers

The values here ($p = 0.30$, $n = 10$, $c = 3$) are illustrative round numbers, chosen so the binomials are small enough to compute on paper — a real HumanEval run uses $n = 200$ samples per problem and averages over 164 problems. What is *not* illustrative is the structure. $\text{pass}@k$ is always 1 minus the probability that every attempt fails; the simple form $1 - (1 - p)^k$ assumes a known p and independent draws *with* replacement, while the estimator $1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$ is the unbiased thing to report from a finite sample, drawn *without* replacement, and it is what real leaderboards ship. Change the constants and the percentages move; the optimistic shape, the saturation, the $(1 - p)$ decay of marginal returns, and the agreement-at- $k=1$ do not. Carry three habits forward: report $\text{pass}@1$ next to any $\text{pass}@k$; use the exact-binomial estimator, not the naive plug-in; and freeze your eval set so the p you measure is capability, not memory.

Appendix A — Reference implementation

Every number in this document is produced by the program below. It uses exact integer binomial coefficients (`math.comb`), so the ratios are exact rationals; change `p`, `n`, `c`, or the `k` range to explore your own benchmark sample.

```

1 # pass_at_k.py -- reproduces every number in "Pass@k, by Hand".
2 from math import comb
3

```

```

4 # ---- Step 1: simple model, pass@k = 1 - (1-p)^k ----
5 p = 0.30
6 print("Simple model p =", p)
7 for k in range(1, 6):
8     val = 1 - (1 - p) ** k
9     print(f" pass@{k} = {val:.4f}")
10
11 # ---- Step 2: marginal gain = p*(1-p)^(k-1), ratio (1-p) ----
12 prev = 0.0
13 print("Marginal gains (geometric, ratio 1-p = 0.7):")
14 for k in range(1, 6):
15     val = 1 - (1 - p) ** k
16     print(f" k={k}: gain = {val - prev:.4f}")
17     prev = val
18
19 # ---- Step 3: unbiased estimator 1 - C(n-c,k)/C(n,k) ----
20 n, c = 10, 3
21 k = 5
22 num = comb(n - c, k) # C(7,5) = 21
23 den = comb(n, k) # C(10,5) = 252
24 unbiased = 1 - num / den
25 print(f"Unbiased n={n} c={c} k={k}: C(7,5)={num} C(10,5)={den}")
26 print(f" pass@{k} = 1 - {num}/{den} = {unbiased:.4f}")
27
28 # ---- Step 4: naive with-replacement vs unbiased ----
29 naive = 1 - (1 - c / n) ** k
30 print(f" naive 1-(1-c/n)^{k} = 1-0.7^{k} = {naive:.4f}")
31 print(f" difference (unbiased - naive) = {unbiased - naive:.4f}")
32
33 # ---- Step 5: unbiased pass@k across k (saturation table) ----
34 print("Unbiased pass@k table n=10, c=3:")
35 for k in range(1, 8):
36     num = comb(n - c, k) if (n - c) >= k else 0
37     den = comb(n, k)
38     print(f" k={k}: C(7,{k})={num:>2} C(10,{k})={den:>3} pass@{k}={1-num/den:.4f}")
39
40 # Expected output:
41 # Simple model p = 0.3
42 # pass@1 = 0.3000
43 # pass@2 = 0.5100
44 # pass@3 = 0.6570
45 # pass@4 = 0.7599
46 # pass@5 = 0.8319
47 # Marginal gains (geometric, ratio 1-p = 0.7):
48 # k=1: gain = 0.3000
49 # k=2: gain = 0.2100
50 # k=3: gain = 0.1470
51 # k=4: gain = 0.1029
52 # k=5: gain = 0.0720
53 # Unbiased n=10 c=3 k=5: C(7,5)=21 C(10,5)=252
54 # pass@5 = 1 - 21/252 = 0.9167
55 # naive 1-(1-c/n)^5 = 1-0.7^5 = 0.8319
56 # difference (unbiased - naive) = 0.0847
57 # Unbiased pass@k table n=10, c=3:
58 # k=1: C(7,1)= 7 C(10,1)= 10 pass@1=0.3000
59 # k=2: C(7,2)=21 C(10,2)= 45 pass@2=0.5333
60 # k=3: C(7,3)=35 C(10,3)=120 pass@3=0.7083
61 # k=4: C(7,4)=35 C(10,4)=210 pass@4=0.8333
62 # k=5: C(7,5)=21 C(10,5)=252 pass@5=0.9167
63 # k=6: C(7,6)= 7 C(10,6)=210 pass@6=0.9667
64 # k=7: C(7,7)= 1 C(10,7)=120 pass@7=0.9917

```

— end of document —